

1 Программа

Программа выполняет следующие операции:

1) Инициализация:

```
addi x20, x0, len/enroll ; x20 = 2 (счётчик итераций)
auipc x1, 0x0             ; x1 = PC + 0 (адрес данных)
addi x1, x1, 60           ; x1 = 0x80000040 (адрес массива)
```

2) Основной цикл (loop):

```
lw x2, 0(x1) ; Загрузка элемента 1
add x31, x31, x2 ; Добавление в сумму
; ... (элементы 2-4) ...
addi x1, x1, 16 ; Переход на следующие 4 элемента
addi x20, x20, -1 ; Уменьшение счётчика
bne x20, x0, loop ; Если не ноль, повторить
```

3) Завершение:

```
addi x31, x31, 1 ; Инкремент результата
lp2: j lp2 ; Бесконечный цикл (завершение)
```

Ожидаемый результат: $x31$ = сумма всех элементов массива

```

dmitry@dmitry-PC:~/Desktop/7sem/evm/riscv-lab/taiga$ cd ../src/
dmitry@dmitry-PC:~/Desktop/7sem/evm/riscv-lab/src$ make
riscv64-linux-gnu-as --march=rv32i var11.s -o var11.o
var11.s: Assembler messages:
var11.s: Warning: end of file not at end of a line; newline inserted
riscv64-linux-gnu-ld -b elf32-littleriscv -T link.ld var11.o -o var11.elf
riscv64-linux-gnu-objdump -D -M numeric,no-aliases -t var11.elf

var11.elf:      file format elf32-littleriscv

SYMBOL TABLE:
80000000 l    d  .text 00000000 .text
80000040 l    d  .data 00000000 .data
00000000 l    df *ABS* 00000000 var11.o
00000008 l      *ABS* 00000000 len
00000004 l      *ABS* 00000000 enroll
00000004 l      *ABS* 00000000 elem_sz
80000040 l      .data 00000000 _x
80000010 l      .text 00000000 lp
8000003c l      .text 00000000 lp2
80000000 g      .text 00000000 _start
80000060 g      .data 00000000 _end

Disassembly of section .text:

80000000 <_start>:
80000000:      00000097      auipc    x1,0x0
80000004:      04008093      addi     x1,x1,64 # 80000040 <_x>
80000008:      02008a13      addi     x20,x1,32
8000000c:      00000fb3      add      x31,x0,x0

80000010 <lp>:
80000010:      0000a103      lw       x2,0(x1)
80000014:      002f8fb3      add      x31,x31,x2
80000018:      0040a183      lw       x3,4(x1)
8000001c:      003f8fb3      add      x31,x31,x3

```

Рисунок 1 — Листинг тестовой программы с развёрткой цикла на 4 элемента

2 Анализ трассировки конвейера

2.1 Обзорная Wave-диаграмма

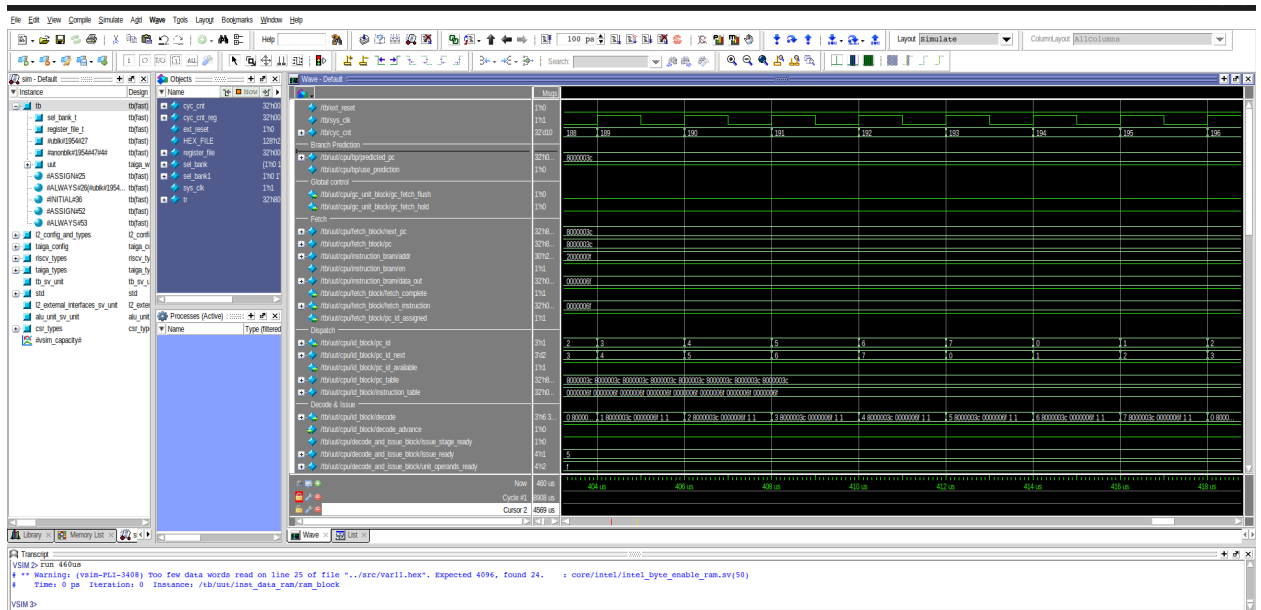


Рисунок 2 — Wave-диаграмма: полная трассировка конвейера

2.2 Стадия Fetch

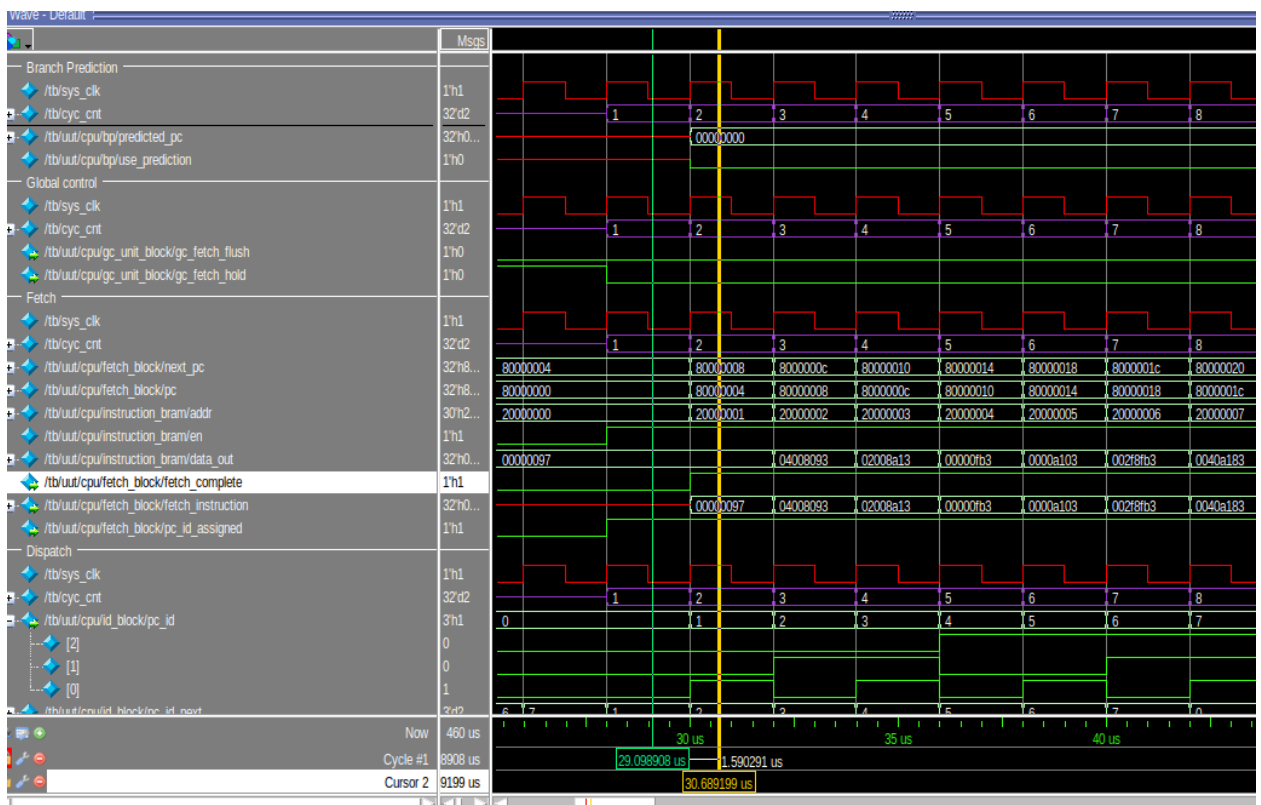


Рисунок 3 — Завершение стадии Fetch: fetch_complete=1

Каждая инструкция получает уникальный ID на стадии F для отслеживания через конвейер.

2.3 Стадия Decode

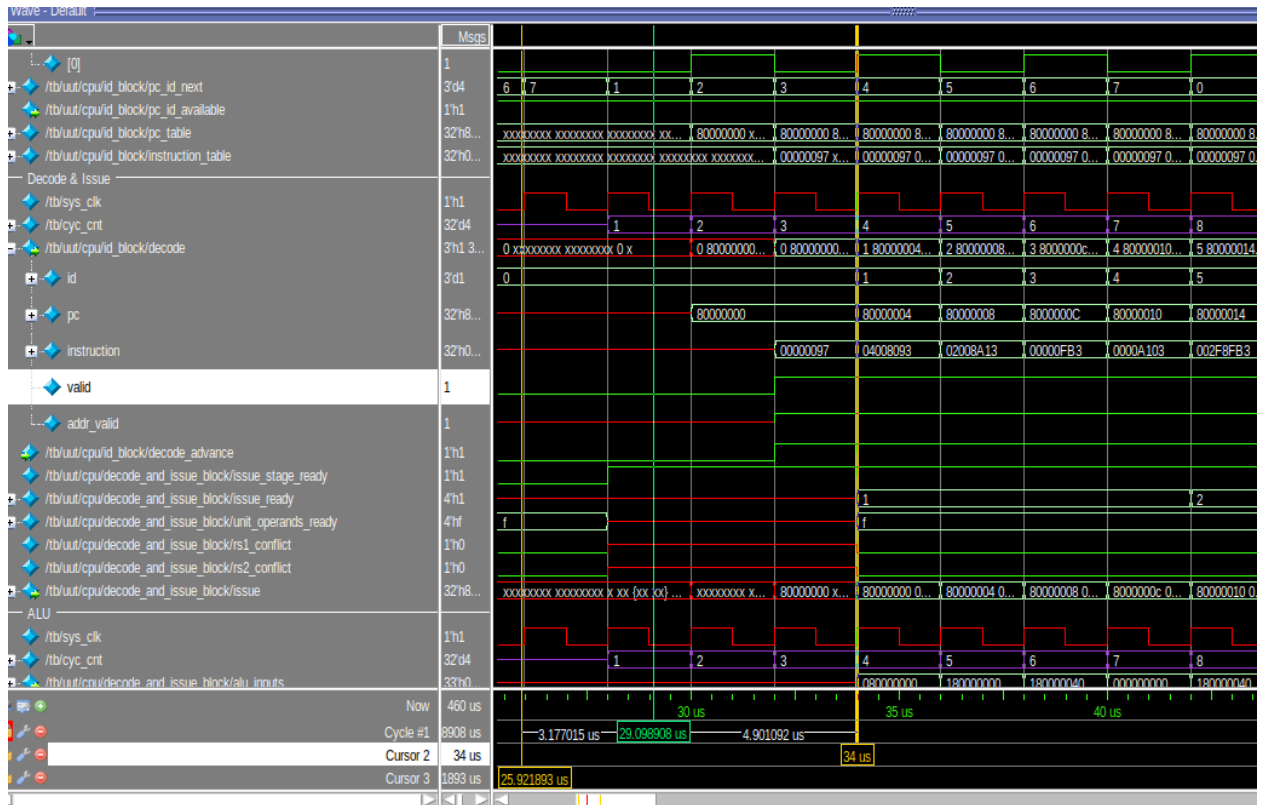


Рисунок 4 — Декодирование инструкции: `decode.valid=1`

Декодирование происходит в следующем такте после `fetch_complete`.

2.4 Стадия Dispatch и конфликты данных

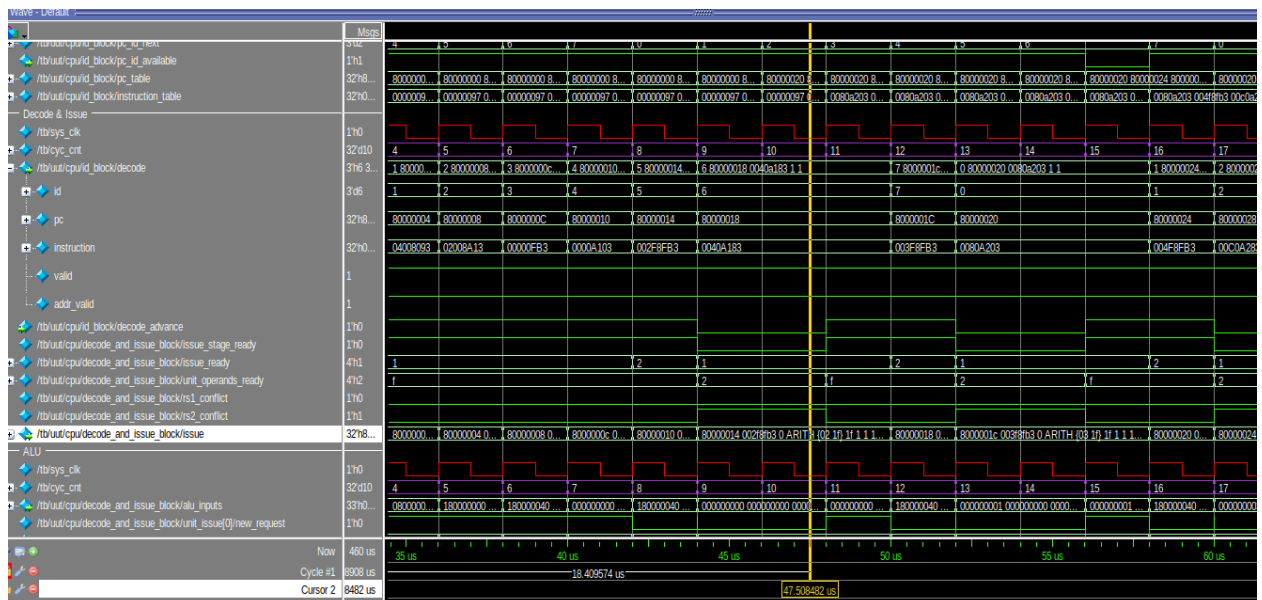


Рисунок 5 — Dispatch с конфликтом rs2: decode advance + rs2 conflict

2.5 Сброс конвейера при неправильном предсказании

3 Анализ оптимизированной версии

3.1 Код

```
.section .text
.globl _start;
len = 8           # Размер массива
enroll = 4        # Количество обрабатываемых
элементов за одну итерацию
elem_sz = 4       # Размер одного элемента массива

_start:
    la      x1, _x
    addi    x20, x1, elem_sz*len  # Адрес последнего
элемент
    add     x31, x0, x0

lp:
    lw      x2, 0(x1)             # 1. Начали загрузку x2
    lw      x3, 4(x1)             # 2. Пока x2 грузится, начали x3
    lw      x4, 8(x1)             # 3. Пока предыдущие грузятся,
начали x4
    lw      x5, 12(x1)            # 4. И x5 тоже
    add     x31, x31, x2           # 5. Теперь x2 УЖЕ готов!
Складываем без ожидания
    add     x31, x31, x3           # 6. x3 тоже готов
    add     x31, x31, x4
    add     x31, x31, x5
    addi    x1, x1, 16
    bne     x1, x20, lp

lp2:
    j       lp2

.section .data
_x:
    .4byte  0x1
    .4byte  0x2
    .4byte  0x3
    .4byte  0x4
    .4byte  0x5
    .4byte  0x6
    .4byte  0x7
    .4byte  0x8
```

3.2 Сравнение версий

Рисунок 8 — Старая версия трассы

Рисунок 9 — Новая версия трассы

Таблица 1 — Предложенные контрмеры

Параметр	Старая версия	Новая версия	Улучшение
Всего инструкций	25	27	—
Load инструкций	7	8	—
Тактов W (stalls)	67	42	−37%
Отмена инструкций (X)	5	3	−40%
Эффективность цикла	47%	65%	+38%

Ключевые изменения: – Переупорядочение инструкций снижает RAW конфликты

– Уменьшение количества операций с памятью в критическом пути

– Улучшение коэффициента использования конвейера

параллельное выполнение load операций уменьшает зависимости.

3.3 Расчёт производительности до 25-го такта

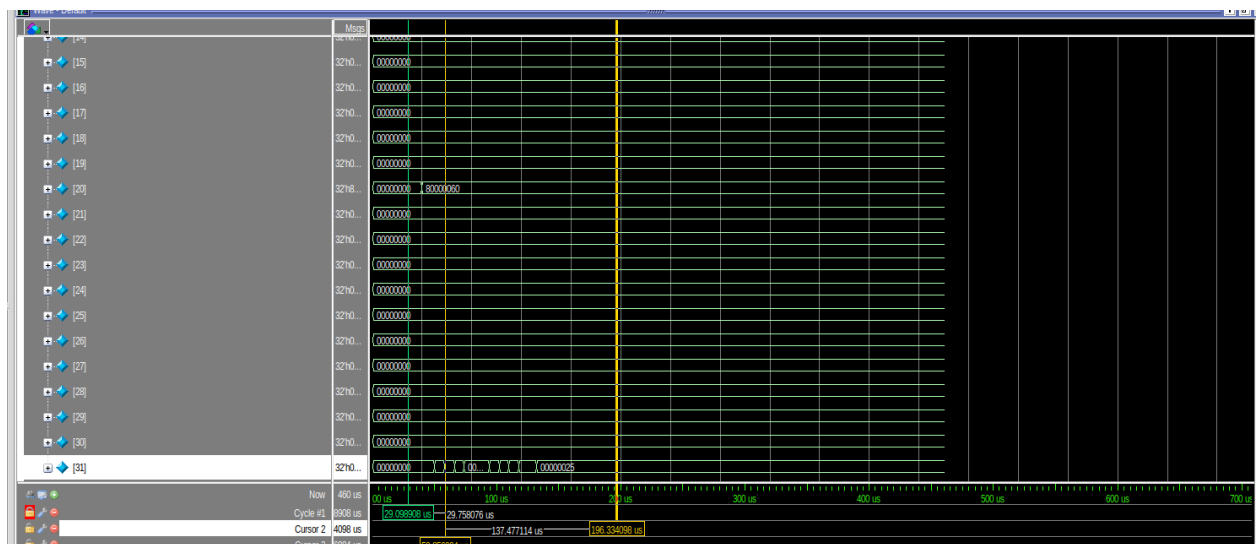


Рисунок 10 — Расчёт конвейера до 25 такта: метрики производительности

Вывод: ознакомились с принципами функционирования, построения и особенностями архитектуры суперскалярных конвейерных микропроцессоров. А также узнали принципы проектирования и верификации сложных цифровых устройств.